

Πότε ένας αλγόριθμος τερματίζει; (Θεωρία Αναδρομής) ¹

Παναγή Αντρέας

Τμήμα Μαθηματικών ΕΚΠΑ

Ιούνιος 2024



ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
Εθνικόν και Καποδιστριακόν
Πανεπιστήμιον Αθηνών

¹andreaspanayi8@gmail.com

- 1 Το πρόβλημα του Τερματισμού
- 2 Απόδειξη μη ύπαρξης αλγορίθμου
- 3 Απεικονιστικές Αναγωγές
- 4 Πέρα από το Πρόβλημα Τερματισμού
- 5 Αριθμητική Ιεραρχία
- 6 Bonus: Θεώρημα Αναδρομής
- 7 Βιβλιογραφία

Θέλουμε να ξέρουμε αν ένας αλγόριθμος (πρόγραμμα) για μια είσοδο τερματίζει.

Για να μελετήσουμε αυτό το πρόβλημα χρειάζεται να το εκφράσουμε στην (αυστηρή) γλώσσα των Μαθηματικών.

Πρέπει δηλαδή, να ορίσουμε την έννοια του *αλγορίθμου* και ένα τρόπο *κωδικοποίησης* (περιγραφής) των αλγορίθμων και των εισόδων τους.

Ορισμός 1 (Διαισθητικός Ορισμός Αλγορίθμου).

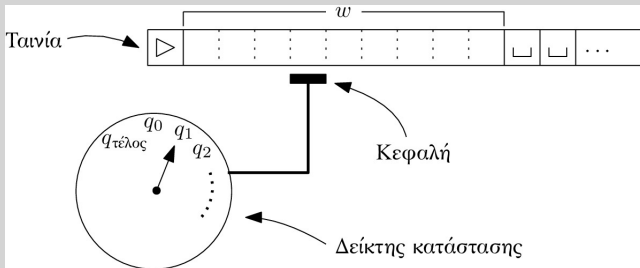
Αλγόριθμος είναι μία πεπερασμένη ακολουθία (αυστηρά καθορισμένων) απλών οδηγιών που διεκπεραιώνουν κάποια εργασία.

Ορισμός 2: Θέση Church-Turing

Μια υπολογιστική διαδικασία ονομάζεται αλγόριθμος αν μπορεί να αναπαρασταθεί από μία *Μηχανή Turing* ή κάποιο ισοδύναμο υπολογιστικό μοντέλο.

(Σχηματική) Αναπαράσταση Μηχανής Turing

Παρότι οι Μηχανές Turing ορίζονται ως μαθηματικά αντικείμενά, δίνουν την δυνατότητα να μπορούν να παρασταθούν σχηματικά (ως μηχανές).



(Μαθηματικά) Μοντέλα Υπολογισμού

Αφού ορίσαμε, ως μαθηματικό αντικείμενο, την έννοια του αλγορίθμου (ως Μηχανή Turing), αξίζει να επισημάνουμε ότι υπάρχουν *ισοδύναμα* (όχι τόσο "μηχανιστικά") πιο "μαθηματικά" μοντέλα όπως οι Αναδρομικές Συναρτήσεις (οι οποίες αποτελούν αφορμή για το όνομα του μαθήματος).

Με άλλα λόγια ο,τι θεωρείτε αλγοριθμικά υπολογίσιμο μπορεί να αναπαρασταθεί ως σύστημα συναρτήσεων - και αντίστροφα.

Σημείωση:

Εκτός από τις Μηχανές Turing και τις Ελαχιστικά Αναδρομικές Συναρτήσεις, ως (ισοδύναμο) μοντέλο θα μπορούσαμε να χρησιμοποιήσουμε τον λ-λογισμό (του Alonzo Church), ένα μοντέλο βασισμένο στη (Μαθηματική) Λογική.

Κωδικοποίηση αλγορίθμων και εισόδων

Τώρα που ορίσαμε την έννοια του αλγορίθμου με μαθηματικό τρόπο, μπορούμε να κωδικοποιήσουμε τους αλγορίθμους (πχ κωδ. Μηχανή Turing) και τις εισόδους, από πεπερασμένες ακολουθίες από 0 και 1.

Συμβολίζουμε με $\langle M \rangle$, την κωδικοποίηση του αλγορίθμου M και ομοίως συμβολίζουμε με $\langle w \rangle$, την κωδικοποίηση της εισόδου w .

Σημείωση:

Κάτι διαισθητικά αντίστοιχο με τις φυσικές γλώσσες π.χ. ελληνικά - όπου χρησιμοποιούμε (σωστά δομημένες) πεπερασμένες ακολουθίες από τα σύμβολα: {α β γ δ ε ζ η θ ι κ λ μ ν ξ ο π ρ σ τ υ φ χ ψ ω}.

Το πρόβλημα του Τερματισμού (Halting Problem)

Θεωρούμε το παρακάτω σύνολο (γλώσσα):

$$HP := \{ \langle M, w \rangle \mid \text{Όπου } \langle M \rangle \text{ κωδικοποίηση αλγορίθμου και } \langle w \rangle \\ \text{κωδικοποίηση της εισόδου τ.ω. ο αλγόριθμος } M \\ \text{με είσοδο } w \text{ τερματίζει} \}$$

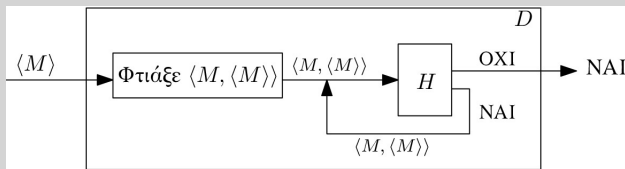
HP είναι δηλαδή το σύνολο που περιέχει κωδικοποιήσεις αλγορίθμων και εισόδων, τέτοιων ώστε ο αλγόριθμος με αυτήν την είσοδο τερματίζει.

Εμείς θέλουμε να ξέρουμε για δοθέν αλγόριθμο και είσοδο αν η κωδικοποίησή τους ανήκει στο σύνολο αυτό ή δεν ανήκει.

Απόδειξη μη ύπαρξης αλγορίθμου

Έστω προς άτοπο ότι υπάρχει αλγόριθμος H που μας λέει ΝΑΙ, αν μια κωδικοποίηση αλγορίθμου και εισόδου ανήκουν στο HP και ΟΧΙ αν δεν ανήκουν.

Κατασκευάζουμε τον παρακάτω αλγόριθμο D ο οποίος δημιουργεί κάτι παράδοξο:



Δίνουμε ως είσοδο του D την κωδικοποίηση του εαυτού του και παρατηρούμε ότι:

Ο αλγόριθμος D με είσοδο τον εαυτό του, τερματίζει αν και μόνο αν δεν τερματίζει!

Αναγωγές: "Δυσκολία" προβλημάτων

Η μη ύπαρξη τρόπου να γνωρίσουμε αν ένα πρόγραμμα με μια είσοδο τερματίζει ή όχι, ανοίγει τον Ασκό του Αιόλου και δημιουργεί διάφορα παρεμφερή ερωτήματα.

Χρησιμοποιώντας ένα (μαθηματικό) "εργαλείο" το οποίο ονομάζεται (απεικονιστική) αναγωγή, εισάγουμε μια έννοια δυσκολίας μεταξύ των προβλημάτων και *αποδεικνύουμε* ότι υπάρχουν κι άλλα "δύσκολα" προβλήματα, όπως και ακόμη δυσκολότερα!

Συμβολίζουμε την αναγωγή του προβλήματος A ("ευκολότερο") στο πρόβλημα B ("δυσκολότερο") ως εξής: $A \leq_m B$

Πέρα από το Πρόβλημα Τερματισμού

Παρεμφερή προβλήματα (που παίρνουμε μόνο το ΝΑΙ) αποτελούν τα εξής:

- (α) Ένας αλγόριθμος με μια είσοδο, γράφει κατά την λειτουργία ένα συγκεκριμένο αριθμό;
- (b) Ένας αλγόριθμος τερματίζει για τουλάχιστον n εισόδους; ($n \in \mathbb{N}$)

Ενώ (κάποια) ακόμη δυσκολότερα (που γενικά δεν παίρνουμε ούτε ΝΑΙ ούτε ΟΧΙ) αποτελούν τα εξής:

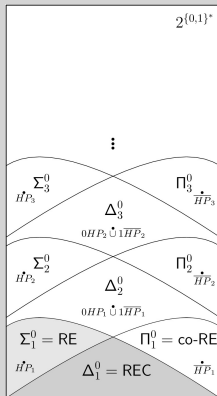
- (i) Ένας αλγόριθμος τερματίζει για το πολύ n εισόδους; ($n \in \mathbb{N}$)
- (ii) Ένας αλγόριθμος τερματίζει για άπειρο το πλήθος εισόδων;

Αριθμητική Ιεραρχία: Κατάταξη προβλημάτων

Τα προβλήματα αυτά κατατάσσονται σε κλάσεις συνόλων τις οποίες ιεραρχούμε, επεκτείνοντας την σχέση δυσκολίας μεταξύ των προβλημάτων.

Η κατάταξη αυτή των κλάσεων ονομάζεται Αριθμητική Ιεραρχία και αποτελεί καρδιά της Θεωρίας Αναδρομής.

Ενδεικτικά:

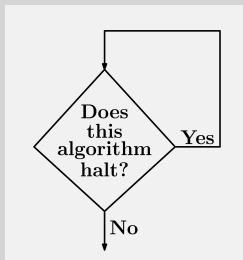


Bonus: Απόδειξη με Θεώρημα Αναδρομής

Το Θεώρημα αναδρομής μας λέει ότι:

Οι αλγόριθμοι μπορούν να χρησιμοποιούν την κωδικοποίηση τους κατά την λειτουργία τους.

Οπότε, με την χρήση αυτο-αναφοράς, οδηγούμαστε στην εξής απλούστερη απόδειξη:



Ο αλγόριθμος τερματίζει αν και μόνο αν δεν τερματίζει.

- [1] Δημήτρης Ζώρος². (Πρόχειρες) σημειώσεις στη ΘΕΩΡΙΑ ΑΝΑΔΡΟΜΗΣ
- [2] Γιάννης Ν. Μοσχοβάκης. ΑΝΑΔΡΟΜΗ ΚΑΙ ΥΠΟΛΟΓΙΣΙΜΟΤΗΤΑ
- [3] Michael Sipser. Εισαγωγή στην Θεωρία Υπολογισμού, Πανεπιστημιακές Εκδόσεις Κρήτης
- [4] Harry R. Lewis, Χρήστος Παπαδημητρίου. Στοιχεία Θεωρίας Υπολογισμού, Εκδόσεις Κριτική

²Ένα μεγάλο ευχαριστώ στον καθηγητή Δ.Ζώρο, για την βοήθειά του στην ολοκλήρωση της παρουσίασης!